Week 8 Examples

Map Demo

This example stores student names and GPAs in an `unordered_map`. It shows inserting and looking up key-value pairs, inspecting hash-table statistics, and comparing unordered-map iteration with the alphabetically sorted order of a `map`.

maps.cpp

```
#include <iostream>
#include <map>
#include <string>
#include <unordered_map>
using namespace std;

void showStats(const unordered_map<string, double>& M)
{
    // n: stored pairs; table's current data size.
    cout << "\nn (number of items): " << M.size() << endl;
    // H: available buckets; more buckets means less crowding.
    cout << "H (bucket count): " << M.bucket_count() << endl;
    // n/H: average pairs per bucket; high means more collisions.
    cout << "load factor (n/H): " << M.load_factor() << endl;
    // Resize threshold; table rehashes past this to stay near O(1).
    cout << "max load factor before resizing: " << M.max_load_factor() <<
endl;
}

int main()
{
    // An unordered_map is a hash table of key-value pairs.
    unordered_map<string, double> gpas;

    gpas.insert({"viviana", 3.9});
    gpas["george"] = 4.0;
    gpas["rob"] = 5.0;
    gpas["will"] = 2.7;
    gpas["santiago"] = 0.5;
    gpas["sean"] = 0.1;

    cout << "Will's GPA is: " << gpas.at("will") << endl;

    cout << "\nUnordered-map items (order and bucket numbers can vary):\n";
    for (const auto& entry : gpas)
        cout << entry.first << " : " << entry.second
             << " : bucket " << gpas.bucket(entry.first) << endl;

    showStats(gpas);
```

```
    // std::map has the same key-value idea, but iterates in sorted-key
order.
    // Insertion and lookup take longer: O(log n)
    map<string, double> sortedGpas(gpas.begin(), gpas.end());
    cout << "\nSame items in std::map order:\n";
    for (const auto& entry : sortedGpas)
        cout << entry.first << " : " << entry.second << endl;
}
```

Graphs · BFS Shortest Path

Problem: Given a directed graph where every edge has the same cost, find the path with the fewest edges from one vertex to another. This example finds shortest paths such as `A -> B -> E -> G` using breadth-first search.

directedGraph.h

```
#pragma once

#include <iostream>
#include <string>
#include <vector>
#include <queue>
#include <unordered_map>
using namespace std;

class directedGraph
{
private:
    class vertex
    {
    public:
        string name;
        vertex* breadcrumb;
        vector<vertex*> neighbors;

        vertex(string x)
        {
            name = x;
            breadcrumb = nullptr;
        }
    };

    // Find a vertex by name. Assumes it was already added.
    unordered_map<string, vertex*> vertexMap;
```

\newpage

```
public:
    void addVertex(string x)
    {
        // Do not replace an existing vertex and lose its edges.
        if (vertexMap.find(x) == vertexMap.end())
            vertexMap[x] = new vertex(x);
    }

    void addDirectedEdge(string from, string to)
    {
        vertex* cur = vertexMap.at(from);
        vertex* nbr = vertexMap.at(to);
        cur->neighbors.push_back(nbr);
    }

    void printAdjacencyList()
    {
        // entry is an iterator with is essentially a key-value pair in the
unordered map
        for (const auto& entry : vertexMap)
        {
            cout << entry.first << ": ";
            for (vertex* neighbor : entry.second->neighbors)
                cout << neighbor->name << " ";
            cout << endl;
        }
    }

    vector<string> shortestPath(string from, string to)
    {
        vertex* start = vertexMap.at(from);
        vertex* goal = vertexMap.at(to);

        // Queue holds vertices waiting to be explored.
        queue<vertex*> q;
        // visited stops one vertex entering the queue twice.
        unordered_map<string, bool> visited;

        // BFS begins at the starting vertex.
        visited[start->name] = true;
        q.push(start);

        while (!q.empty())
        {
            // Take the oldest vertex from the queue.
            vertex* cur = q.front();
            q.pop();

            // First visit to goal uses the fewest edges.
            if (cur == goal)
                break;
```

```
            addNeighbors(cur, q, visited);
        }

        // An unvisited goal cannot be reached from start.
        if (!visited[goal->name])
            return {};

        return backtrack(start, goal);
    }

    void addNeighbors(vertex* cur, queue<vertex*>& q, unordered_map<string,
bool>& visited)
    {
        // Add every new neighbor to the back of the queue.
        for (vertex* neighbor : cur->neighbors)
        {
            if (!visited[neighbor->name])
            {
                visited[neighbor->name] = true;
                // Save this vertex to rebuild the path later.
                neighbor->breadcrumb = cur;
                q.push(neighbor);
            }
        }
    }

    vector<string> backtrack(vertex* start, vertex* goal)
    {
        vector<string> path;
        // Walk backward: goal, parent, grandparent, ..., start.
        vertex* cur = goal;
        while(cur != start)
        {
            path.push_back(cur->name);
            cur = cur->breadcrumb;
        }
        path.push_back(start->name);

        // It was built backward, so swap matching ends.
        int length = path.size();
        for (int i = 0; i < length / 2; i++)
        {
            int mirror = length - 1 - i;
            swap(path[i], path[mirror]);
        }

        return path;
    }
};
```